QALight

Frontend-розробка

Заняття 16. Об’єктна модель документа

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний



Об’єктна модель документа. Модель об’єкта браузера.
Дерево DOM. Об’єкт document. Об’єкт консолі.
Властивості елементів. Доступ до властивостей.
Методи пошуку елемента. Властивість HTMLElement.style. Маніпулювання атрибутами елемента.
Об’єкт navigator. Об’єкт location.
Зміна атрибутів знайдених елементів в об'єкті document.

Модель об’єкта браузера

Об’єкт navigator. Об’єкт location.

Модель об’єкта браузера (Browser Object Model, BOM)

Теорія: BOM

Модель об’єкта браузера (Browser Object Model, BOM) - це набір об’єктів, які надає браузер для взаємодії з вікном браузера та середовищем виконання, за винятком самого вмісту вебсторінки.

Головним об'єктом BOM є window, який одночасно представляє вікно браузера та глобальний об'єкт у JavaScript. Об'єктна модель документа (Document Object Model, DOM) є частиною BOM через властивість window.document, яка відповідає за вміст сторінки.

BOM не має єдиного офіційного стандарту (як DOM у W3C), проте сучасні браузери підтримують схожий набір базових API. Основні складові та об'єкти BOM:

- window - глобальне вікно, що містить методи для керування розмірами (innerWidth, innerHeight), таймерами (setTimeout, setInterval) тощо.
- navigator - інформація про сам браузер та операційну систему користувача (наприклад, версія, мова, наявність мережі).
- location - управління URL поточної сторінки (дозволяє перенаправляти користувача на іншу адресу або дізнатися поточний шлях).
- history - історія переходів користувача в поточному вкладці (методи back(), forward(), go()).
- screen - інформація про параметри екрана пристрою (ширина, висота, глибина кольору).

Об'єкт window

Теорія: BOM

Об'єкт window - це найважливіший об'єкт у JavaScript для браузерів. Він є глобальним об'єктом, що означає дві речі:

- він представляє саме вікно браузера
- слугує контейнером для всіх глобальних змінних, функцій та інших об'єктів BOM.

Вікно для конкретного документа можна отримати за допомогою властивості `document.defaultView`.

У коді JavaScript доступна глобальна змінна `window`, яка представляє вікно, де виконується скрипт.

Інтерфейс Window містить різноманітні функції, простори імен, об'єкти та конструктори, які не обов'язково безпосередньо пов'язані з поняттям вікна користувацького інтерфейсу.

У браузері з вкладками кожна вкладка представлена власним об'єктом `Window`; глобальний об'єкт `window`, доступний для коду JavaScript у межах певної вкладки, завжди представляє саме ту вкладку, де виконується цей код. Водночас, навіть у браузері з вкладками, деякі властивості та методи стосуються загального вікна, що містить вкладку (наприклад, `resizeTo()` та `innerHeight`). Загалом, усе, що не стосується безпосередньо вкладки, відноситься до вікна.

Керування розмірами та прокручуванням вікна

Теорія: BOM

Глобальна змінна window дозволяє дізнатися розміри видимої області (**viewport**) та керувати положенням сторінки:

- window.**innerWidth** / window.**innerHeight** - повертають поточну ширину та висоту контентної області вікна (включаючи смуги прокручування).
- window.**scrollTo**(x, y) - прокручує сторінку до конкретних координат (наприклад, window.scrollTo({ top: 0, behavior: 'smooth' }) для плавного повернення нагору).
- window.**scrollBy**(x, y) - прокручує сторінку відносно її поточного положення.

```
console.log(window.innerWidth, window.innerHeight);
window.scrollBy(900, 500);
window.scrollTo({ top: 0, behavior: 'smooth' }) ;
```

Діалогові вікна

Теорія: BOM

Браузер надає три прості блокуючі (зупиняють виконання коду) методи для взаємодії з користувачем:

- alert(message) - виводить інформаційне повідомлення з кнопкою "ОК".
- confirm(question) - виводить запитання з кнопками "ОК" та "Скасувати". Повертає true або false.
- prompt(text, default) - показує вікно з полем введення тексту. Повертає введене значення або null.

```
iif (window.confirm("Ви хочете очистити кошик?")) {
    let userName = window.prompt("Введіть ваше ім'я для підтвердження:");
    window.alert(`Дякуємо, ${userName}, кошик очищено.`);
}
```

(Префікс window. писати не обов'язково, можна просто викликати alert()).

Робота з часом

Теорія: BOM

Усі функції для роботи із затримками та інтервалами належать об'єкту window:

- setTimeout(callback, delay) - виконує функцію один раз після закінчення вказаного часу (у мілісекундах).
- setInterval(callback, interval) — регулярно виконує функцію через кожні N мілісекунд.
- clearTimeout(id) / clearInterval(id) — зупиняють роботу відповідного таймера за його ідентифікатором.

```
// Запуск таймера
const timerId = window.setTimeout(() => {
  console.log("Минуло 3 секунди!");
}, 3000);
// Скасування таймера за потреби
window.clearTimeout(timerId);
```

Глобальний контекст виконання

Теорія: BOM

Будь-яка змінна, оголошена через var у глобальній області видимості, або функція, створена стандартним шляхом, автоматично стає властивістю об'єкта window:

```
var globalVar = "Привіт";

function sayHello() { return "Тест"; }

console.log(window.globalVar); // "Привіт"
console.log(window.sayHello()); // "Тест"
```

(Сучасні ключові слова **let та const створюють змінні в глобальній області, але не додають їх до об'єкта window** для уникнення забруднення глобального простору).

Об'єкт location

Теорія: BOM

Об'єктна модель браузера (BOM) не обмежується лише window. Інші ключові об'єкти дозволяють керувати адресою сторінки, переміщатися по історії, дізнаватися інформацію про пристрій та систему користувача.

Об'єкт location (Керування адресою та навігацією) містить інформацію про поточну **URL-адресу** сторінки та дозволяє перенаправляти користувача.

Основні властивості (для розбору URL):

- location.href - повна URL-адреса сторінки (наприклад, https://example.com).
- location.protocol - протокол (http: або https:).
- location.host / location.hostname - доменне ім'я з портом або без (example.com:8080 / example.com).
- location.pathname - шлях до сторінки (/search).
- location.search - параметри запиту (?q=js).
- location.hash - якір/хеш (#result).

Методи для навігації

Теорія: BOM

Основні методи (для навігації):

- location.assign(url) - завантажує нову сторінку (зберігає поточну в історії, можна повернутися назад).
- location.replace(url) - замінює поточну сторінку новою (видаляє поточну з історії, кнопка "Назад" не поверне сюди).
- location.reload() - перезавантажує поточну сторінку.

```
// Перенаправлення на інший сайт із можливістю повернутися
window.location.href = "https://google.com";
// Зміна лише параметру хешу (сторінка не перезавантажиться)
window.location.hash = "#contacts";
```

Об'єкт history

Теорія: BOM

Об'єкт history (Історія переглядів вкладки) - надає доступ до історії сесії браузера (у межах конкретної вкладки) і дозволяє програмно імітувати натискання кнопок "Назад" та "Вперед".

- history.back() - повертає на одну сторінку назад (аналог кнопки ⬅️ в браузері).
- history.forward() - переходить на одну сторінку вперед (аналог кнопки ➡️).
- history.go(n) - переходить на n сторінок вперед (якщо n додатне) або назад (якщо n від'ємне). Наприклад, history.go(-2) поверне на дві сторінки назад.
- history.length - кількість сторінок в історії поточної сесії.

```
console.log("history.length: ", history.length);
```

(з міркувань безпеки JavaScript не може бачити самі URL-адреси сайтів, які користувач відвідував раніше, лише переміщатися між ними).

Об'єкт navigator

Теорія: BOM

Об'єкт navigator містить дані про сам веб-переглядач, операційну систему, стан мережі та доступні апаратні можливості.

- navigator.userAgent — рядок із інформацією про назву браузера, його версію та ОС (часто використовується для аналітики).
- navigator.language — мова інтерфейсу браузера (наприклад, uk-UA чи en-US).
- navigator.onLine — логічне значення (true / false), яке показує, чи підключений пристрій до інтернету прямо зараз.
- navigator.clipboard — сучасне API для читання та запису в буфер обміну (копіювання тексту).
- navigator.geolocation — дозволяє запитати у користувача дозвіл на отримання його геокоординат.

```
if (navigator.onLine) {
  console.log("Ви підключені до мережі!");
} else {
  alert("Інтернет-з'єднання втрачено. Збережіть роботу.");
}
```

Об'єкт screen

Теорія: BOM

Об'єкт screen надає інформацію про фізичні властивості монітора або дисплея пристрою, на якому запущено браузер.

- screen.width / screen.height — загальна ширина та висота фізичного екрана в пікселях.
- screen.availWidth / screen.availHeight — доступна ширина та висота екрана (за вирахуванням системних елементів, таких як панель завдань Windows або Dock на macOS).
- screen.colorDepth — глибина кольору монітора (кількість біт на піксель).

```
// Перевірка, чи це великий монітор, чи екран мобільного
console.log(`Роздільна здатність екрана: ${screen.width}x${screen.height}`);
console.log(`Доступна висота для вікна: ${screen.availHeight}px`);
```

Об'єктна модель документа (DOM)

Дерево DOM. Об’єкт document. Об’єкт консолі. Властивості елементів. Доступ до властивостей. Методи пошуку елемента. Властивість HTMLElement.style. Маніпулювання атрибутами елемента.

Об'єктна модель документа (DOM)

Теорія: DOM

Об'єктна модель документа (DOM) - це програмний інтерфейс (API), який дає змогу розробникам отримувати доступ до структури та вмісту HTML- або XML-документів. DOM являє собою документ у вигляді ієрархічного дерева елементів, де кожен елемент є об’єктом.

Браузер читає HTML-код і створює модель документа, де кожен тег, атрибут чи шматок тексту стає окремим об'єктом-вузлом.

Об’єктна модель документа дає змогу розробникам створювати інтерактивні та динамічні веб-сторінки. За допомогою DOM можна знаходити елементи на сторінці, додавати, видаляти та змінювати елементи документа, а також керувати їхнім стилем і поведінкою.

Будь-які зміни в об'єктній моделі миттєво відображаються браузером на екрані користувача.

У 2004 році W3C (World Wide Web Consortium) опублікувало стандарт DOM Level 3, що включає багатий набір API для роботи з документами.

Основні можливості DOM

Теорія: DOM

Основні можливості DOM:

Зміна вмісту елементів (textContent, innerHTML).

Маніпуляція стилями та класами (style, classList).

Обробка подій (кліки миші, натискання клавіш, введення тексту).

Додавання та видалення елементів на сторінці.

Структура DOM

Теорія: DOM

Модель DOM представляє структуру HTML або XML-документа у вигляді деревоподібної ієрархії. Кожен елемент у документі, такий як теги HTML або XML, представляється вузлом у цьому дереві. Вузли встановлюють зв’язки батько-потомок, утворюючи ієрархію елементів.
HTML є кореневим елементом.

```
<!-- HTML Document Structure -->
<html>
```

Head і Body - дочірні елементи HTML.

```
<!-- Document Head -->
 <head>
   <!-- Document Title -->
   <title>"Page Title"</title>
 </head>
 <!-- Document Body -->
 <body>
```

Title, Header, Main, і Footer - дочірніми елементами Head і Body відповідно. Текстові вузли (наприклад, «Заголовок сторінки» і «Привіт, світ!») являють собою вміст конкретних елементів.

Дерево елементів

Теорія: DOM

Дерево елементів - це структура, яка відображає вкладеність HTML-тегів один в одного. Вона будується за принципом «родинних зв'язків» між вузлами:

- Батьківський елемент (Parent): Елемент, всередині якого містяться інші. Наприклад, якщо всередині лежить <p>, то є батьком для <p>.
- Дочірній елемент (Child / Children): Елемент, розташований безпосередньо всередині іншого.
- Сестринські елементи (Siblings): Елементи, які мають одного й того самого «батька» (лежать на одному рівні вкладеності).

Вузол (Node)

Теорія: DOM

У DOM вся HTML-сторінка представляється у вигляді ієрархічного дерева елементів, де кожна окрема деталь є вузлом (node).

Вузол (Node) - це базовий будівельний блок DOM-дерева. Абсолютно все, що є в HTML-файлі, стає вузлом. Проте вони бувають різних типів.

Основні типи вузлів:

- Вузол елемента (Element Node): Це самі HTML-теги (, <p>, <h1>, <a>). Вони складають основу структури сторінки.
- Текстовий вузол (Text Node): Будь-який текст всередині тегів (навіть звичайні пробіли та перенесення рядків браузер перетворює на текстові вузли).
- Вузол коментаря (Comment Node): Коментарі, які ви залишаєте в коді на зразок <!-- Це коментар -->, теж потрапляють у DOM-дерево.
- Вузол документа (Document Node): «Корінь» усього дерева (об'єкт document), який представляє всю сторінку загалом.

Інтерфейс document

Теорія: DOM

Інтерфейс **document** описує загальні властивості та методи для будь-якого типу документа. Кожна веб-сторінка в браузері має власний об’єкт **document**. Інтерфейс **document** є **точкою входу** для отримання вмісту веб-сторінки та забезпечує глобальну функціональність. До об’єкту можна звертатись так **window.document або document**.

Скалярні властивості об’єкта document загальні у всіх браузерах. Значення властивостей кольора текста, фона і посилань можна змінювати динамічно лише у тих випадках, коли **не надані відповідні властивості CSS**, що мають більший пріоритет.

- ittle - текст заголовка документа (title);
- fgColor і bgColor - колір текста і фона документа;
- linkColor, vLinkColor, aLinkColor - колір посилань;
- lastModified (тільки читати) - дата модифікації документа;
- referrer (тільки читати) - URL документа, посилання в якому призвело до завантаження поточного документа;
- URL - URL документа.

Метод консолі браузера console.dir()

Теорія: DOM

Метод API консолі браузера **console.dir()**, використовується для реєстрації об’єктів JavaScript на консолі. За допомогою **console.dir()** ми отримуємо інтерактивний список з всіма доступними атрибутами та функціями для HTML-елемента.

- Властивість **tagName** повертає ім’я тегу у верхньому регістрі. Наприклад, якщо є елемент HTML , tagName поверне «DIV».
- Властивість **nodeName** для HTML-елементів повертає ім’я тегу у **верхньому регістрі,** для інших типів вузлів повертає рядок, що представляє тип вузла (наприклад, "**#text**" для текстових вузлів, "**#comment**" для вузлів коментарів).
- Кожен вузол у DOM має властивість **nodeType**, яка вказує його тип і має числове значення: 1 для елементів, 2 для атрибутів, 3 для текстових вузлів, 8 для коментаря та 9 для документа. Цю властивість можна використовувати для розрізнення вузлів елементів і текстових вузлів, наприклад:

```
const element = document.body;
console.log(element.nodeType); // Output: 1
```

Властивості документа

Теорія: DOM

- Властивість **doctype** повертає тип документа (DTD). Властивість doctype є властивістю лише для читання: `const doctypeObj = document.doctype;`
- Властивість **characterSet** (лише для читання) повертає кодування символів документа.
- Властивість **`document.head`** повертає елемент <head> поточного документа:
  console.log(document.head); // head
- Властивість **`documentURI`** повертає розташування документа у вигляді рядка:
  console.log(document.URL); // http://localhost:1234/

Де-які властивості DOM-елементів можна читати та змінювати, інші - тільки читати.

```
console.log(document.domain); // localhost
console.log(document.URL); // http://localhost:1234/
console.log(document.title); // Shopping cart
console.log(document.doctype); // <!DOCTYPE html>
console.log(document.head); // head
```

Доступ до властивості документа

Теорія: DOM

Доступ до властивості відбувається по ключу за допомогою крапкової нотації через **об'єкт document:** console.log(document.title); // title
Доступ до властивості, що задана не валідним JavaScript-ідентифікатором (містить пробіл, тире або), можна отримати за допомогою **квадратних дужок**: console.log(document['title']); // title

- Властивість **елемента tagName** містить ім'я тага у верхньому регістрі, **тільки читати**: console.log(document.body.tagName); // BODY
- Властивість **document.childElementCount** повертає кількість дочірніх елементів документа: console.log(document.childElementCount);
- Властивість **document.firstElementChild** повертає перший дочірній елемент документа.
- Властивість **document.lastElementChild** повертає останній дочірній елемент документа або null, якщо дочірніх елементів немає: console.log(document.lastElementChild);
- Всі дочірні елементи, включаючи текстові, містяться у **масиві childNodes**: console.log('Всі дочірні елементи ', document.body.childNodes);

Методи DOM

Теорія: DOM

JavaScript пропонує низку методів для доступу до елементів DOM та їхньої маніпуляції. Деякі з ключових методів включають:

- getElementById(id): отримує елемент за його ідентифікатором.
- getElementsByClassName(className): повертає колекцію елементів з певним класом.
- getElementsByTagName(tagName): повертає колекцію елементів з певним тегом.
- querySelector(selector): повертає перший елемент, що відповідає заданому селектору CSS.
- querySelectorAll(selector): повертає всі елементи, що відповідають заданому селектору CSS.

Метод getElementById(elementID)

Теорія: DOM

Метод **getElementById(elementID)** повертає елемент документа, шо має вказаний ідентифікатор (id="elementID"), у вигляді об’єкта Node (вузла). Якщо елемент не існує, метод поверне значення **null**.

Властивість id має бути унікальною в межах документа. **Всі ідентифікатори id чутливі до регістру**. За допомогою даного методу можна отримати доступ до будь-якого елемента у документі, що має атрибут ID:

```
<h1 id="hell">Hello JavaScript!</h1>
var hello = document.getElementById('hell');
```

Метод **querySelector()** повертає перший знайдений елемент в документі (для **document**) або серед дочірніх вузлів (для **node**), що відповідає CSS селектору, вказаному в якості параметра метода.

Даний метод має один **обов’язковий параметр** (**selector**), який є рядком, що містить CSS селектор. **element = document.querySelector(selectors);**

- element - об’єкт element.
- selectors - рядок, що містить один або більше CSS селекторів.

Метод querySelector застосовують, коли потрібно знайти один конкретний елемент. Метод повертає тільки **перше співпадіння, або null**. document.querySelector('h1').textContent = 'Hello JavaScript!';

Властивість HTMLElement.style

Теорія: DOM

Властивість **HTMLElement.style** повертає об’єкт **CSSStyleDeclaration**, що описує атрибут стилю елемента. Властивість **style** має той же пріоритет, що і пряма декларація стилю через **атрибут style**. **Краще** використовувати style ніж elt.setAttribute('style', '...'), бо використання властивості **style не буде перезаписувати інші CSS властивості** атрибуту style.

```
document.body.style.backgroundColor = 'red'; // Можна змінити колір BODY
```

Використовується **camelCase**: **background-color**: document.body.style.**backgroundColor.** Виключенням є властивість **float**: **elem.style.cssFloat замість elem.style.float.** Слово **float** є зарезервованим у стандарті JavaScript, і його використання у стильових властивостях неможливе. Стилі з **браузерним префіксом**, наприклад, **-webkit-border-radius** перетворюються аналогічно: button.style.**WebkitBorderRadius** = '5px';

Отримати або встановити атрибут style у вигляді рядка можна за допомогою властивості **cssText**. Властивість **cssText** дозволяє задати кілька CSS стилей одним рядком. Весь **вміст атрибута style перезаписується**.

```
document.body.style.cssText = "color: #072a38; font: 300 18px/1.6 'Source Sans Pro',sans-serif; margin: 0; padding: 5em 0 2em; text-align: center;";
```

Властивість innerHTML

Теорія: DOM

Властивість innerHTML в JavaScript дозволяє отримувати або змінювати повний HTML-код, який міститься всередині елемента).
Властивість innerHTML застосовують, в основному, для динамічних змін контексту сторінки:
Ви можете використовувати **innerHTML** для доступу або зміни вмісту HTML всередині елемента.
document.body.innerHTML = '<h1>Bye! See ya!</h1>';
На відміну від textContent та innerText, які працюють виключно з чистим текстом, innerHTML взаємодіє безпосередньо з HTML-розміткою та змушує браузер парсити й рендерити теги.
Ключові особливості innerHTML

- Працює з тегами: При зчитуванні повертає рядок, що містить усі вкладені HTML-теги, атрибути та тексти. При записі - перетворює рядок тексту на реальні DOM-елементи.
- Ризик безпеки (XSS-атаки): Якщо ви вставляєте в innerHTML дані, які ввів користувач (наприклад, з форми коментарів), зловмисник може впровадити шкідливий скрипт.
- Перерендер вмісту: Щоразу, коли ви змінюєте innerHTML (навіть через оператор +=), браузер повністю видаляє всі старі елементи всередині вузла і створює нові. Це скидає встановлені через JavaScript слухачі подій (addEventListener) на вкладених елементах.

Зчитування та запис вмісту

Теорія: DOM

```
// Приклад зчитування вмісту
<div id="content">
 <p>Привіт, <strong class="highlight">світ</strong>!</p>
</div>

const content = document.getElementById('content');
console.log(content.innerHTML);// Виведе точну копію HTML-коду:// "<p>Привіт, <strong
class="highlight">світ</strong>!</p>"

// Приклад запису вмісту
const list = document.getElementById('content');
// Вставимо новий список всередину div
list.innerHTML = `
 <ul>
   <li>Елемент 1</li>
   <li>Елемент 2</li>
 </ul>
`; // Браузер миттєво згенерує маркований список на сторінці
```

Головне правило безпеки: Уникайте XSS

Теорія: DOM

```
// Головне правило безпеки: Уникайте XSS
// Якщо ви отримали текст від користувача і запишете його в innerHTML, це може призвести до виконання стороннього коду:

const userInput = "<img src='x' onerror='alert(\"Ваш сайт зламано!\")'>"; // НЕБЕЗПЕЧНО:
element.innerHTML = userInput; // Скрипт виконається через onerror!
// БЕЗПЕЧНО:
element.textContent = userInput; // Відобразить це просто як текст


// Властивість outerHTML дозволяє отримати або встановити HTML-вміст елемента включаючи сам елемент.
```

Властивість innerText

Теорія: DOM

Властивість innerText призначена для отримання або зміни видимого текстового вмісту елемента та його нащадків. Вона поводиться так, ніби користувач виділив текст на сторінці мишкою та скопіював його. Якщо елемент або його батько мають стилі display: none або visibility: hidden, innerText не поверне цей текст, оскільки він прихований від користувача.
Властивість innerText автоматично зчитує теги
 або блочні елементи (як-от <p>,) і перетворює їх на справжні переноси рядків (\n). Властивість прибирає зайві пробіли, підлаштовуючи текст під те, як він реально відображається у браузері. Кожен виклик властивості innerText змушує браузер перераховувати макет сторінки, що працює повільніше за textContent.

```
<div id="demo">
 Перший рядок.
 <p style="display: none;">Цей текст сховано за допомогою CSS.</p>
 <div>Другий рядок.</div>
</div>
const demo = document.getElementById('demo');
console.log(demo.innerText);
// (Схований текст повністю ігнорується, а div створив перенос рядка)
```

Коли ви записуєте значення в innerText, воно видаляє всі вкладені теги та вставляє переданий рядок як чистий текст.

```
const box = document.getElementById('demo');
box.innerText = "Привіт, \n це новий рядок!";
// На сторінці з'явиться два рядки тексту, а символ \n спрацює як тег <br>
```

Властивість textContent

Теорія: DOM

Властивість textContent використовується для отримання або встановлення текстового вмісту вузла та всіх його нащадків.

- Повертає текст як самого елемента, так і всіх вкладених у нього тегів.
- Теги всередині елемента не перетворюються на HTML, а відображаються як звичайний текст.
- На відміну від innerHTML, textContent є безпечним від атак типу XSS (Cross-Site Scripting), бо не виконує переданий код чи теги.
- Працює швидше за innerText, оскільки не зважає на стилі CSS і не змушує браузер перераховувати макет сторінки (reflow).

```
<div id="box">Привіт, <span>світ</span>!</div>
-
// Отримання тексту:
const box = document.getElementById('box');
console.log(box.textContent); // Виведе: "Привіт, світ!"
// Зміна тексту:
const box = document.getElementById('box');
box.textContent = 'Новий текст без тегів <b>жирний</b>';
// Усі теги відображаться як текст на сторінці
```

Чим textContent відрізняється від innerText

Теорія: DOM

Властивості textContent, innerText та innerHTML мають різні підходи до читання та запису вмісту HTML-елементів. Головна відмінність полягає в тому, що innerHTML працює з HTML-розміткою, textContent повертає сирий текст прямо з дерева DOM, а innerText відображає лише видимий користувачу текст, враховуючи стилі CSS.

Зчитування вмісту

```
<div id="example">
 Привіт,
 <span style="display: none;">секретний</span>
 <b>світ!</b>
</div>
```

element.**innerHTML** поверне рядок разом із тегами та стилями:
"Привіт, секретний світ!"

element.**textContent** поверне весь текст, включно із прихованим, ігноруючи самі теги:
"Привіт, секретний світ!"

element.**innerText** поверне лише те, що реально бачить користувач на екрані (приховане слово зникне, а переноси рядків підлаштуються під зовнішній вигляд): "Привіт, світ!"

Чим textContent відрізняється від innerText при записі

Теорія: DOM

Різниця при записі (Встановленні значення)

Коли ви передаєте рядок з HTML-тегами (наприклад, 'Новий текст'), властивості поводяться так:

innerHTML - створить тег жирного тексту. Користувач побачить: Новий текст.

textContent та **innerText** - екранують теги. Користувач побачить на сторінці сирий текст: Новий текст.

Властивість classList

Теорія: DOM

Для управління окремими класами елемента використовується властивість **classList** – об'єкт з методами для додавання/видалення класу. Властивість **classList** повертає масив **DOMTokenList**, що містить всі класи елемента. Об'єкт **classList** має властивість **length**, що повертає кількість класів у елемента. Управління класами елемента здійснюється за допомогою методів об'єкта DOMTokenList:

- add – додати клас;
- remove – видалити клас;
- toggle – переключити клас;
- contains – перевірити наявність класу у елемента.

Для **classList** існує примітивна альтернатива - властивість **className**, що містить значення атрибута **class** елемента.

```
var cName = elem.className;
elem.className = cName;
```

cName - рядок. Кілька класів вказуються через пробіл.

Маніпуляції з атрибутом class

Теорія: DOM

Додати новий атрибут класу до елемента можна за допомогою методу **add**() об'єкта classList:
Element.classList.add('myClass');

Видалити клас можна за допомогою методу **remove**():
Element.classList.remove('myClass');

Щоб замінити існуючий клас новим, скористайтеся методом **replace**():
Element.classList.replace('oldClass', 'newClass');

Метод **toggle**() працює як перемикач: додає клас, якщо його немає, і видаляє клас, якщо він є.
Element.classList.toggle('myClass');

Щоб перевірити, чи містить елемент певний клас, скористайтеся методом **contains**() і передайте клас, який ви хочете перевірити, як рядок:
Element.classList.contains('myClass');
Метод повертає значення true, коли вказано клас. В іншому випадку він повертає false.

Метод setAttribute()

Теорія: DOM

Якщо ви хочете змінити інші атрибути, такі як id, href або src, ви можете скористатися методом **setAttribute**(). Метод setAttribute() приймає два аргументи:

- Ім'я атрибута, який потрібно встановити
- Значення атрибута, яке потрібно встановити

Наприклад, ось як встановити атрибут src тегу img:

```
img.setAttribute('src', 'new-image.jpg');
```

Якщо ви хочете отримати значення атрибута, ви можете скористатися методом **getAttribute**().

Якщо атрибут встановлено, метод повертає значення цього атрибута у вигляді рядка. Якщо ні, повертається null:

```
img.getAttribute('src'); // нове зображення.jpg
img.getAttribute('href'); // нуль
```

Якщо ви хочете видалити атрибут, скористайтеся методом **removeAttribute**():

```
img.removeAttribute('src');
```

Пошук елементів на сторінці (сучасні методи)

Теорія: DOM

Пошук елементів на сторінці - це процес, під час якого JavaScript звертається до DOM-дерева, щоб знайти потрібний вузол (або групу вузлів) за певними критеріями (ідентифікатором, класом, тегом або CSS-селектором).
Сучасний JavaScript пропонує кілька способів вирішення цього завдання, які можна розділити на дві основні групи: універсальні сучасні методи та класичні методи.
Універсальні сучасні методи використовують синтаксис звичайних CSS-селекторів (тих самих, які ви пишете у файлах стилів .css). Вони є найбільш гнучкими та популярними в сучасній веб розробці.
document.**querySelector**('селектор'): Шукає та повертає перший знайдений елемент, який відповідає селектору. Якщо нічого не знайдено, повертає null.

```
const mainTitle = document.querySelector('#main-title'); // за id
const activeLink = document.querySelector('.nav__link.active'); // за класами
const firstParagraph = document.querySelector('div > p'); // за вкладеністю тегів
```

document.**querySelectorAll**('селектор'): Знаходить усі елементи на сторінці, що відповідають селектору, і повертає їх у вигляді колекції (статичного списку NodeList).

```
const allButtons = document.querySelectorAll('.card button');// Отриману колекцію можна легко перебрати через цикл
```

Класичні методи

Теорія: DOM

Класичні методи швидкі, але вузькоспеціалізовані. Вони шукають елементи лише за одним конкретним параметром і не підтримують складні CSS-комбінації.

- document.getElementById('id') - шукає один унікальний елемент за його id. Працює дуже швидко.
- document.getElementsByClassName('class') - знаходить усі елементи з таким класом. Повертає «живу» колекцію HTMLCollection.
- document.getElementsByTagName('tag') - знаходить усі елементи з відповідним тегом (наприклад, усі <a> чи <img>).

Якщо ви знайшли елементи через getElementsByClassName, а потім за допомогою JS видалили один із них зі сторінки, ця колекція автоматично зменшиться в розмірі.

Методи querySelectorAll повертають статичний «знімок» сторінки на момент пошуку, який сам по собі не змінюється.

Практика

Змінити вміст і стилі HTML-елементу на сторінці

Додавання обробника події (кліку) на кнопку

Практика: DOM

Ось простий та наочний приклад того, як за допомогою JavaScript знайти HTML-елемент на сторінці та змінити його вміст і стилі.

HTML-розмітка (Початковий стан)

Уявімо, що на сторінці є заголовок із унікальним ідентифікатором (id) та кнопка:

```
<h1 id="main-title">Привіт, Світе!</h1>
<button id="change-btn" onclick="doit()">Змінити заголовок</button>
```

Змінити вміст і стилі HTML-елементу на сторінці

Практика: DOM

```
function doit() {
    // 1. Пошук елементів на сторінці
    const title = document.getElementById('main-title');
    const button = document.getElementById('change-btn');

    // 2. Зміна текстового вмісту елемента title
    title.textContent = 'Текст успішно змінено через DOM!';

    // 3. Зміна стилів елемента title (робимо колір зеленим)
    title.style.color = '#27ae60';
    // 4. Зміна стилів елемента button
    button.style.backgroundColor = "red";
    button.style.color = "white";
    // 5. Зміна текстового вмісту елемента button
    button.textContent = "All done!";
  };
```

ВПРАВИ

Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Консорціум Всесвітнього павутиння https://www.w3.org/

ECMAScript https://ru.wikipedia.org/wiki/ECMAScript

ECMA http://www.ecma-international.org/

Standard ECMA-262 http://www.ecma-international.org/publications/standards/Ecma-262.htm

JavaScript Web APIs:

- https://www.w3.org/standards/webdesign/script
- https://developer.mozilla.org/en-US/docs/Glossary/JavaScript

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua